ROOM//ZERO

END-TO-END ARCHITECTURE AND 8-PERSON BUILD PLAN

Meta VR Start Developer Competition 2026 • Team execution guide

Core promise: Your real room becomes the game engine. ZERO creates a room-aware heist, orchestrates specialist agents, and remembers player choices across sessions.

BUILD TARGET

A stable Quest prototype: scan/understand one room, place spatial objectives, complete one playable hand-and-voice heist, adapt once to player behavior, and persist the result.

Prepared from the uploaded ROOM//ZERO concept brief. The six-agent concept, room-based heist, hand/voice-first play, adaptation, and persistent memory are retained. Implementation details in this guide are recommended engineering choices, not claims that they are already built.

1. Product target and MVP boundaries

ROOM//ZERO is an MR heist experience built around a real physical room. The concept brief describes seven player-facing beats: ask ZERO for a mission; analyze the room; instantiate a holographic heist; use hands and voice; solve puzzles and make choices; have ZERO learn; return to a changed mission later.

For a hackathon, prove the core loop rather than attempting a limitless game platform. The smallest convincing vertical slice is one room, one mission template, two interactive objectives, one NPC moment, one adaptation event, and a saved player profile.

MUST SHIP	SHOULD SHIP	DEFER IF TIME IS SHORT
Quest build launches; passthrough/room understanding; one safe playable heist; hand interaction; voice start/choice; ZERO mission orchestration; saved session summary.	Two mission variations; basic spatial audio; NPC reaction; difficulty adjustment based on retries/time; clear onboarding and reset.	Fully autonomous open-ended world generation; complex multi-NPC combat; cloud-scale analytics; online multiplayer; unrestricted LLM-controlled physics or object placement.

> **Definition of done:** a teammate unfamiliar with the project can install the build, enter a clear play area, start a mission, complete it without a controller, restart the app, and see evidence that the prior choice affected the next session.

Assumptions to confirm on day one

• Target device is a Meta Quest headset with passthrough and hand tracking supported by the chosen Unity/Meta XR SDK version.

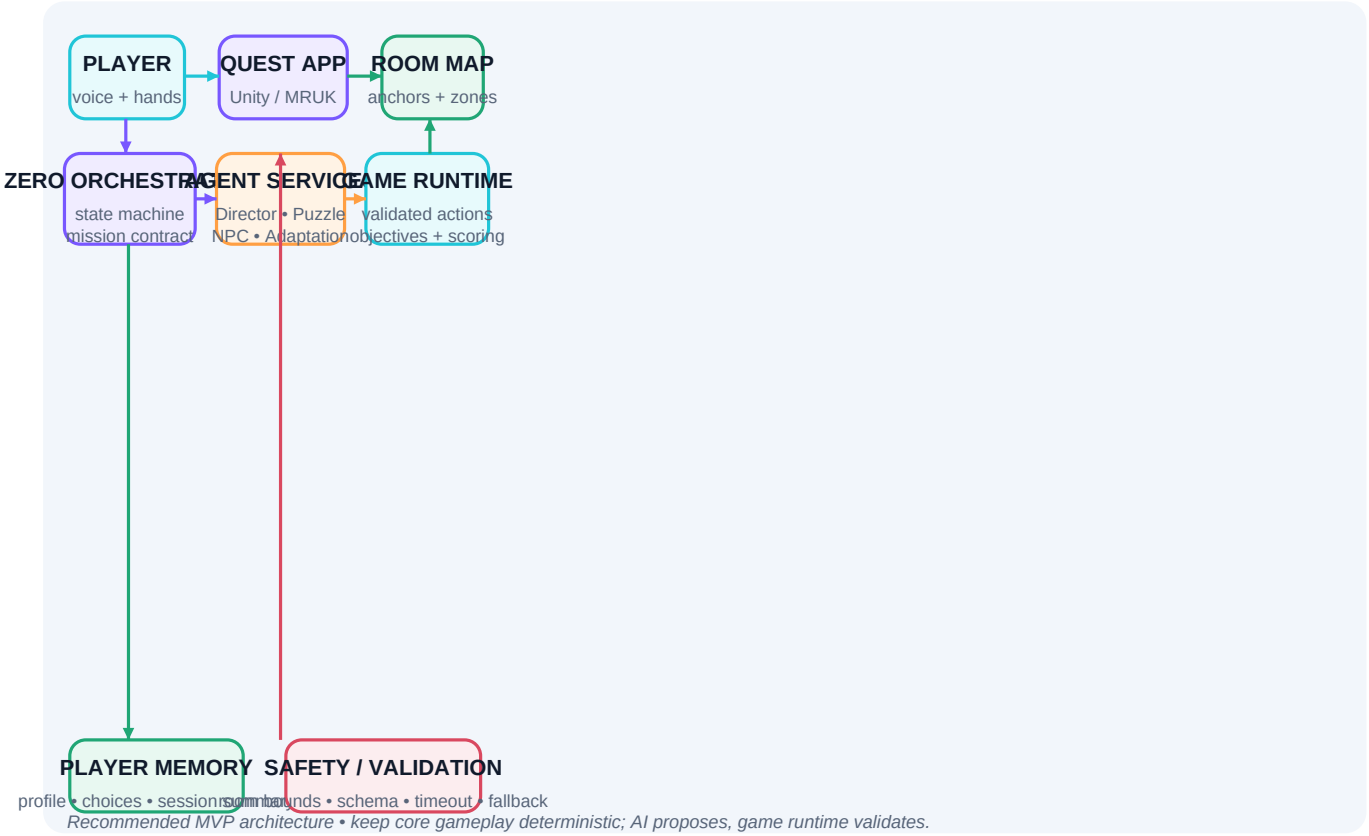
• Choose one supported room-understanding path early (for example, Meta MR Utility Kit / scene API). Do not let two people independently build competing room-scanning approaches.

• Use a local deterministic fallback for every AI-dependent action so a network/model failure never blocks the demo.

• Only use permitted competition SDKs, models, assets, and APIs; verify current competition rules separately.

2. System architecture

Recommended division: Unity owns spatial rendering, interaction, mission state, scoring, and safety. ZERO orchestrates mission planning and specialist agents. AI output is treated as a proposal; the deterministic game runtime validates it before anything appears or changes in the room.



Recommended MVP architecture • keep core gameplay deterministic; AI proposes, game runtime validates.

Runtime responsibilities

Layer	Owns	Contract / output
Quest / Unity client	Passthrough, room surfaces, anchors, hand input, voice capture, UI, audio, game visuals.	Normalized events: MissionRequested, HandAction, VoiceIntent, ObjectiveCompleted.
Room Scout	Convert supported scene/room data into a compact safe room map.	RoomMap: surfaces, bounds, zones, confidence, excluded areas.
ZERO orchestrator	Own mission lifecycle, agent routing, timeouts, retries, fallbacks and state transitions.	MissionPlan + validated GameCommand list.
Specialist agents	Director, Puzzle Architect, NPC Agent, Player Model, Adaptation Agent; Room Scout supplies spatial context.	Structured JSON only; no direct Unity scene writes.
Game runtime	Validate command schema and spatial bounds; spawn safe objects; update objectives, timers and score.	Authoritative MissionState and events.
Memory store	Save compact player profile, choices and session summaries; version data.	ProfileSnapshot used for next mission.

3. End-to-end runtime sequence

01 • Boot

Load Unity scene, permissions/onboarding, input systems, and local fallback mission. Confirm headset and hand tracking status.

02 • Room Scout

Read supported room/scene data. Normalize surfaces and free zones into a RoomMap. If scan data is missing, offer a bounded fallback layout or ask the player to rescan.

03 • Mission request

Player says “ZERO, give me a mission” or selects a simple start affordance. Create a session ID and snapshot the player profile.

04 • Plan

Director proposes objective/theme/difficulty. Puzzle Architect maps puzzle placements to valid surfaces. NPC Agent returns dialogue/behavior beats. Adaptation Agent sets initial challenge level.

05 • Validate

Schema validator checks required fields, object count, difficulty limits, placement bounds, reachable interaction zones and forbidden locations. Reject or repair invalid plans; use a deterministic template after repeated failure.

06 • Play

Unity spawns holographic props at anchors. Hand/voice events enter the game runtime. The runtime—not the LLM—decides whether a puzzle is solved, whether a guard notices, and whether the mission succeeds.

07 • Adapt

Observe simple signals: elapsed time, repeated failed attempts, hints requested, and objective success. Adjust one variable at a time (hint, timer, guard alertness, or puzzle step).

08 • Persist

At mission end, save choices, completion, time, retries, hint use, and a short summary. Update profile features such as preferred puzzle style and recent difficulty.

09 • Return

Next session reads the saved profile and recent world state, then changes at least one meaningful mission element while keeping room safety and interaction rules fixed.

4. Six brain centers: scope and safe interfaces

Agent	Input	Output	Guardrail
Room Scout	Room/scene data, room bounds, surface labels	Normalized RoomMap	Never infer hidden hazards; mark unknown confidence; exclude doors, stairs, edges and blocked zones.
Game Director	RoomMap summary, profile, mission catalog	MissionPlan: theme, objective IDs, difficulty, duration	Choose only supported mission templates/objective types.
Puzzle Architect	Valid surfaces/zones, puzzle catalog, difficulty	PuzzleSpec list with target anchor IDs and parameters	No arbitrary coordinates or unbounded object spawning.
NPC Agent	Mission state, character persona, recent dialogue	Dialogue line + enumerated behavior intent	No free-form command execution; short lines; fallback dialogue.
Player Model	Session events and past summaries	Profile features: pace, retries, hints, preferences	Store game telemetry only; avoid unnecessary personal data.
Adaptation Agent	Current performance signals, current difficulty	One bounded adjustment proposal	Clamp ranges; cool-down between adjustments; never make safety harder.

Important engineering rule

Agents should not be six independent free-running chat loops. Use one orchestrator with explicit states, timeouts, structured inputs/outputs, and a deterministic mission state machine. Invoke only the specialist needed for the current step; this reduces latency and makes failures explainable.

Suggested state machine

BOOT → ROOM_READY → MISSION_REQUESTED → PLANNING → VALIDATING → BRIEFING → ACTIVE → ADAPTING (optional) → COMPLETE → SAVING → READY_FOR_NEXT_SESSION. Every state must have a timeout and an error/fallback transition.

5. Data contracts and example payloads

Keep agent output small and typed. JSON is illustrative; implement equivalent C# serializable classes and validate every response before use.

RoomMap (normalized)

```
{ "schemaVersion": 1, "roomId": "local-room-01", "surfaces": [
{"id":"surface-01","type":"table","center":[0.4,0.75,-0.6],
"size":[0.8,0.03,0.5],"confidence":0.92,"allowed":true} ], "zones": [
{"id":"safe-zone-01","type":"playable","bounds":"normalized-bounds"} ], "excludedZoneIds":
["door-zone","edge-zone"] }
```

MissionPlan (validated proposal)

```
{ "schemaVersion": 1, "missionId": "heist_001", "templateId": "DESK_VAULT", "difficulty": 1,
"durationSeconds": 180, "objectives": [ {"id":"obj_1","type":"FIND_CLUE","anchorId":"surface-01"},
{"id":"obj_2","type":"ENTER_CODE","targetId":"vault-panel"} ], "npcBeatIds":
["guard_intro","guard_react"], "successCondition": "ALL_OBJECTIVES_COMPLETE" }
```

Player profile / session summary

```
{ "schemaVersion": 1, "profileId": "local-player", "stats": {"missions": 2, "completionRate": 0.5,
"medianSeconds": 145, "hintRate": 0.3}, "preferences": {"puzzleStyle":"observation"}, "lastSession":
{"missionId":"heist_001","choice":"SILENT_ROUTE", "completed":true,"retries":1} }
```

Implementation notes

• Version each contract (`schemaVersion`) and reject unknown required fields.

• Use enums for mission templates, objective types, NPC intents, and adaptation actions.

• Do not accept arbitrary executable code, shell commands, direct file paths, or raw transforms from an AI response.

• Persist local-first for demo reliability; add cloud sync only if already proven and allowed.

6. Eight-person team assignment

Give every person one accountable subsystem and a concrete integration artifact. Names are intentionally role-based; replace them with your team members' names at kickoff.

# / Role	Owns	Deliverables	Depends on / handoff
1. Tech Lead + Unity Integrator	Unity project structure, architecture decisions, build pipeline, main scene, integration gates.	Runnable Quest build; event bus/interfaces; integration checklist; daily build.	Defines interfaces with all roles; merges only tested branches.
2. MR / Room Understanding Engineer	Meta XR setup, passthrough, room/scene data, anchors, safe zones.	RoomMap provider; visible surface markers/debug view; no-scan fallback.	Hands normalized RoomMap to #1 and #3.
3. Interaction + VR UX Engineer	Hand tracking interactions, voice input, onboarding, diegetic UI, feedback.	Hand-only select/grab/press; mission-start voice; accessible fallback UI.	Publishes normalized input events to #1/#5.
4. AI Orchestration Engineer	ZERO state machine, agent routing, schemas, timeouts, retries, fallback.	Mission orchestration API; deterministic template fallback; trace logs.	Coordinates contracts with #5 and game commands with #1.
5. Agent / LLM Engineer	Director, Puzzle Architect, NPC, Player Model, Adaptation prompts and structured outputs.	Six brain-center adapters; test fixtures; prompt/version notes.	Uses contracts from #4; cannot mutate scene directly.
6. Game Systems + Puzzle Designer	Heist loop, objective logic, guard/stealth rules, win/fail states, balancing.	One complete mission; two puzzle interactions; deterministic scoring.	Defines objective catalog and GameCommand enum for #1/#4/#5.
7. Persistence + Player Model Engineer	Local save schema, session summary, profile updates, migration/reset.	Save/load test; next-session change; sample profile fixtures.	Consumes mission events from #1/#6 and exposes profile to #4/#5.
8. QA + Build/Submission + Demo Producer	Device test matrix, performance checks, bug triage, install guide, 3-minute video/story.	Acceptance checklist; known-issues log; release build; demo script/video.	Tests every integrated build; owns final submission packaging, not core feature code.

If someone has a stronger specialty, swap owners—but keep one accountable owner per subsystem and keep QA independent enough to challenge the build.

7. Build order and milestone gates

The source brief proposes a three-week progression. For a shorter hackathon, preserve the dependency order and compress time—not the order. Do not build six sophisticated agents before the first mission works.

Gate	Work	Owner(s)	Exit criteria
G0 • Day 1	Lock SDK/version/device, repository, scene, branch rules, contracts, mission concept.	1 + all	Quest app opens on target device; all team members can build; one agreed mission template.
G1 • Foundation	Passthrough/room data, hand tracking, voice start, debug room map.	1, 2, 3	Room-ready scene; at least one valid surface/zone represented; input events visible in logs.
G2 • Deterministic loop	Spawn objectives, hand interaction, clue/code puzzle, win/fail state.	1, 3, 6	Mission completable without AI/network; stable reset; safety boundaries enforced.
G3 • ZERO + agents	Orchestrator calls mission planner and specialist agents; validate response; fallbacks.	4, 5, 6	AI selects among supported mission variations; invalid/late response still starts fallback mission.
G4 • Memory/adaptation	Save choice/performance, reload profile, adjust one difficulty variable.	5, 7	Restart app and verify previous choice changes the next briefing/mission variation.
G5 • Release	Polish, performance, regression, clean build, capture video, rehearse pitch.	1, 8 + all	Three successful full runs on target hardware; video shows actual gameplay; backup build/video ready.

Daily cadence

• 10-minute stand-up: yesterday / today / blocker. Report blockers with a reproducible symptom.

• Two integration checkpoints daily. Feature branches must merge to main only when the project builds.

• Freeze new features before final demo rehearsal. Reserve at least 20% of remaining time for integration and device-specific bugs.

8. Repository layout and integration rules

Suggested layout (adapt to the existing repo rather than reorganizing late):

```
ROOM-ZERO/ UnityProject/ Assets/RoomZero/ Scenes/ Prefabs/ Materials/ Audio/ Scripts/ App/ # bootstrap, scene flow, event bus Spatial/ # room provider, anchors, safe zones Input/ # hand, voice, normalized events Game/ # mission state, objectives, scoring Zero/ # orchestrator, agent adapters, validators Memory/ # profile, session store, migrations UI/ # onboarding, briefing, HUD, results Tests/ docs/ architecture.md contracts.md device-test-checklist.md demo/ script.md capture-checklist.md
```

Branch and code ownership rules

• Main must always be buildable. Use feature branches and short pull requests.

• Agree on shared contracts before parallel coding: `RoomMap`, `MissionPlan`, `GameCommand`, `MissionEvent`, `PlayerProfile`.

• Only the Unity/game runtime spawns or moves objects. Agent adapters return proposals.

• Keep API secrets out of Git. Use local config ignored by Git and a documented mock mode.

• Use a single logging convention with session ID, state name, agent name, duration, and fallback reason.

Minimal internal APIs

Call / event	Purpose	Failure behavior
RoomProvider.GetRoomMap()	Return normalized geometry/surfaces/zones.	Return explicit unavailable status; activate safe fallback layout or rescan.
ZeroDirector.CreateMission(request)	Return validated mission plan.	Timeout → deterministic mission template.
GameRuntime.ApplyCommand(command)	Spawn allowed object / advance objective / play NPC beat.	Reject invalid enum, out-of-bounds placement, stale mission ID.
MemoryStore.Save(summary)	Persist completion, choices and performance signals.	Queue retry or keep in-memory session; never block mission end.

9. Spatial safety, privacy, and reliability

This is a room-scale MR experience. Safety is a system requirement, not a late polish task.

Risk	Required mitigation
Player walks into furniture / walls	Keep interactions reachable from a stable standing position where possible; never require running or blind movement; place virtual targets inside validated zones; provide pause/exit.
Incorrect room geometry	Use confidence and allowed flags; reject low-confidence surfaces; show a brief scan/room check; use a bounded fallback rather than guessing.
AI invents unsafe placement or actions	AI returns anchor IDs and enums, not arbitrary transforms or executable code. Runtime validates room bounds, distance, count, and supported action type.
Network/model failure	Timeout, retry once if time permits, then use deterministic local plan/dialogue. Mission must remain completable offline after boot.
Unexpected AI text	Validate length and content; use short NPC lines; provide silent/visual equivalent; never make voice the only route for a critical action.
Sensitive room data	Prefer local processing and local persistence for the demo. Do not upload room meshes, photos, or identifiers unless necessary, disclosed, and permitted.
Tracking/comfort issue	Test seated/standing assumptions, recenter behavior, hand tracking loss, lighting variation, and a clear emergency pause/quit action.

Acceptance tests (run on the target headset)

Test	Pass condition
Clean launch	App launches from a fresh install; permissions and onboarding are understandable.
Room scan absent / poor	User receives clear guidance or safe fallback; no invalid object placement.
Hand tracking lost	Game pauses or explains how to recover; no accidental objective completion.
Voice unavailable/noisy	On-screen or hand-operated alternative can start and complete the mission.
Malformed/slow agent output	Plan is rejected; fallback mission starts within agreed timeout.
Out-of-bounds anchor	Runtime rejects it and logs a reason; no object spawns there.
Persistence	Complete a mission, close/relaunch app, and verify saved choice/profile is loaded.
Adaptation	Repeated failure or fast completion changes exactly one bounded challenge variable.
Regression run	Three consecutive end-to-end runs without crash, stuck state, or manual editor intervention.

10. Three-minute judge demo plan

The concept brief proposes a real-room opening, a transformation, hand puzzles, NPC interaction, visible adaptation, mission completion, and a changed next session. Capture real headset gameplay; use title cards only to bridge unavoidable setup time.

Time	What to show	Proof point
0:00–0:10	Show the actual room and player: “ZERO, give me a mission.”	Voice-first entry into the experience.
0:10–0:25	Show room surfaces/anchors and the mission briefing appearing in context.	Spatial context is used, not a generic flat VR level.
0:25–1:00	Use hands to reveal a clue and enter a code / complete two objectives.	Playable deterministic loop and controller-free interaction.
1:00–1:35	NPC reacts to a player choice; show a short voice or subtitle response.	Agent output changes a game beat.
1:35–2:10	Intentionally struggle or succeed quickly; show one bounded adaptation such as a hint or guard alertness change.	Adaptation uses observed signals and affects gameplay.
2:10–2:35	Complete mission and show a concise saved choice/session summary.	Persistent player memory is real and inspectable.
2:35–3:00	Start a second mission/session and show a changed briefing/objective while the room remains the same.	Reason to return: the same room yields a different run.

Demo reliability checklist

• Have a clean, charged headset; clear the play area; test lighting and tracking in the exact demo room.

• Record one successful full run in advance as a backup. Label it honestly as recorded gameplay if used.

• Prepare a local mock AI mode and a known-good mission template; do not depend on live network response for the main path.

• Ensure the video visibly proves the claims: hand interaction, room-aware placement, one AI-driven change, and memory after restart.

11. Map engineering proof to judging criteria

Criterion in brief	What to demonstrate	Evidence / metric
Innovation	Mission uses actual room surfaces/anchors and produces a room-specific objective.	Show room map + placement; explain why it is not a prebuilt flat level.
Experience design	Hands and voice lead the interaction; visual feedback and comfort controls are clear.	Complete objectives without controllers; test voice fallback and hand tracking loss.
Technical	Orchestrator routes structured agent proposals to deterministic gameplay; profile changes next session.	Architecture slide, logs, schema validation, persistence test.
Polish	Stable frame pacing, readable holograms, spatial audio, clean transitions and no stuck state.	Three repeatable runs; bug list closed; clean capture.

Top project risks and countermeasures

Risk	Early warning	Countermeasure
Overbuilding multi-agent AI	No playable mission by first integration gate.	Ship one deterministic mission first; agents select and tune from supported options.
SDK/room data surprises	Team cannot obtain stable surfaces/anchors on target headset.	Prototype room provider immediately; create mock RoomMap for parallel gameplay work.
Integration collision	Multiple people edit same Unity scenes/contracts.	One Unity integrator; agreed interfaces; scene ownership; daily main build.
AI latency / network	Briefing takes too long or API is unavailable.	Short structured requests, timeout, cached/local fallback, asynchronous loading UI.
Memory claim not visible	Saved data exists but next mission looks identical.	Persist one meaningful choice and make it visibly alter the next briefing/objective.

12. Kickoff checklist and ownership board

Complete these decisions before splitting into parallel work. Ambiguity here creates expensive rework later.

- ■ Confirm target Quest device(s), Unity version, Meta XR SDK/MRUK version and required permissions.
- ■ Run a tiny test scene on hardware: passthrough + hand tracking + read one room surface/anchor.
- ■ Select the single MVP mission: e.g. desk vault with clue → code panel → guard reaction → escape/complete.
- ■ Agree the contract types and enum values before agent and gameplay code diverge.
- ■ Agree AI mode: live model/API if allowed and reliable, plus deterministic mock/fallback mode.
- ■ Create repository, branch rules, ignored secrets/config, build instructions and owner list.
- ■ Set two daily integration times and a feature freeze point.
- ■ Schedule at least three full end-to-end headset runs and one recorded backup demo.

Ownership board (fill in during kickoff)

Subsystem	Owner name	Backup	First delivery / due
Unity integration / build	_____	_____	_____
Room understanding / anchors	_____	_____	_____
Hand + voice UX	_____	_____	_____
ZERO orchestration	_____	_____	_____
AI agents / prompts	_____	_____	_____
Puzzle + game systems	_____	_____	_____
Persistence / player model	_____	_____	_____
QA + demo / submission	_____	_____	_____

Final principle

> Build the experience as a game first, then let AI make it personal. The room map constrains what can happen; the orchestrator coordinates specialists; the deterministic runtime decides what actually happens; memory makes the next session meaningfully different.

Source basis: uploaded “ROOM//ZERO Team Brief” (8 pages). This plan preserves its concept and stated six brain centers, three-minute video beats, and broad phase progression, while adding recommended engineering contracts, ownership, testing, safety, and integration detail for an eight-person team.